Welcome to the final video for Domain 3 in this theory-only series for the AWS Machine Learning Engineer Associate exam. We have chosen deployment targets and provisioned resources. Now we automate everything with continuous integration and continuous delivery, often called CI/CD, and MLOps practices. In this video we cover the AWS developer tools, SageMaker Pipelines, versioning for models, prompts, and agents, safe deployment strategies with rollback, automated testing, and automated retraining and knowledge base refresh. Let's begin.

CI/CD applies proven software practices to machine learning. Everything starts in source control. Code, pipeline definitions, infrastructure templates, and even prompts live in Git. AWS CodeConnections links AWS to providers like GitHub, GitLab, and Bitbucket. AWS CodeCommit is AWS's own Git service and still appears on the exam, although new customers now usually use an external provider through CodeConnections. A commit triggers the pipeline. In the build and test stage, AWS CodeBuild runs the steps listed in a buildspec dot yml file. It runs unit tests and data checks and builds container images that it pushes to Amazon ECR. In the release stage, AWS CodePipeline connects all stages, adds manual approval gates, and deploys through AWS CloudFormation or AWS CodeDeploy, moving from development to staging to production. If a pipeline fails, the first place to look is the CodeBuild logs and the IAM permissions of each stage.

CodePipeline handles the software side. For the ML workflow itself, Amazon SageMaker Pipelines is the key service. A pipeline is a series of steps. First, a processing step prepares and validates the data. Second, a training step runs the training job with the instance type, hyperparameters, and data channels you configured. Third, an evaluation step computes metrics on held-out test data. Fourth, a condition step checks those metrics against a threshold, such as accuracy above a target or error below a limit. If the model fails the check, the pipeline stops. If it passes, the fifth step registers the model as a new version in the SageMaker Model Registry. Because every step is defined in code, each run is repeatable and fully tracked. On the exam, when a question asks for automated, repeatable training with a quality gate, SageMaker Pipelines with a condition step is usually the answer.

If you cannot say exactly which model, prompt, and agent version produced an answer, you cannot audit or reproduce it. The SageMaker Model Registry organizes models into model package groups, with a new version for each registered model. Each version has an approval status. Changing it to approved can trigger an Amazon EventBridge rule that starts deployment. Managed MLflow on SageMaker AI tracks experiments, logs parameters and metrics, compares runs, and registers the best models. For generative AI, prompts change behavior just like code. Amazon Bedrock Prompt Management stores prompts as versions with variables and model settings, so you can compare variants and pin a tested version in production. Bedrock agents use versions and aliases. Your application calls an alias, and rolling back simply means pointing the alias to the previous version. Fine-tuned foundation models become custom models, so tag each one with its training data and settings.

Replacing a production model all at once is risky. SageMaker AI deployment guardrails let you shift traffic gradually. With blue/green deployment, a new green fleet runs next to the old blue fleet. Traffic can move all at once, as a canary, where a small share goes first, or linearly, in equal steps. A rolling deployment updates the existing capacity in batches, which helps for large endpoints where doubling capacity would be expensive. Rollback is what makes this safe. You attach Amazon CloudWatch alarms, for example on errors or latency. After each traffic shift there is a baking period. If an alarm fires during that time, SageMaker AI automatically sends traffic back to the old fleet. Always keep the previous model version

approved so you have something to return to. Exam tip: minimize risk to users usually means canary or linear with automatic rollback.

Automated tests inside the pipeline catch problems before users do. For traditional ML, test the data first: schema checks and data quality rules. Then run unit and integration tests on the code. Finally, compare the new model's metrics to the current production baseline, and block the release if it is worse. Generative AI needs a different kind of test. Keep a golden dataset of questions with expected answers and run it every time a prompt or model changes. This is prompt regression testing. Amazon Bedrock model evaluation jobs can score responses automatically, and LLM-as-a-judge scales the review, but spot-check it with humans, as we discussed in Domain 2. Agents need end-to-end tests. Check that the agent picks the right tool, completes multi-step scenarios correctly, and respects guardrails. A pipeline that fails fast is far cheaper than a bad model in production.

Models and knowledge go stale, so updates should be automatic. Retraining starts with a trigger. It might be a schedule in Amazon EventBridge, a drift alarm from SageMaker Model Monitor, or new labeled data arriving in Amazon S3. The trigger starts the same SageMaker pipeline we saw earlier. Using the same pipeline and the same evaluation keeps results comparable. The new model is registered as a new version and must be approved before it is deployed. RAG systems need a similar loop. When source documents change, the knowledge base must be synced. You can run ingestion jobs on a schedule, or use S3 events to start a sync when files change. After every refresh, rerun your retrieval tests to confirm quality has not dropped. A common exam scenario describes a chatbot giving outdated answers, and the fix is usually an automated knowledge base sync, not retraining the model.

Let's finish Domain 3 with this checklist. One: store code, pipeline definitions, and prompts in version control. Two: build, test, and deploy through CodePipeline stages with CodeBuild. Three: register every model, prompt, and agent version so you can audit and roll back. Four: gate releases on automated tests for data, metrics, prompts, and agents. Five: roll out gradually with blue/green, canary, or linear traffic shifting, and use CloudWatch alarms for automatic rollback. Six: automate retraining and knowledge base refresh so the system stays current. That completes Domain 3. Across these three videos, we went from choosing deployment targets, to provisioning resources, to automating the full lifecycle. Next, we move on to Domain 4: operating, monitoring, and securing ML and AI solutions. Thanks for watching, and see you in the next video.